

ЗВІТ

до лабораторної роботи № 5.2

на тему:

"Практичне використання Aggregation Framework у MongoDB."

Виконала:

студентка групи МІТ-31

Йовхимищ Діана

Мета роботи: закріпити знання про основні стадії Aggregation Framework. Навчитися будувати ефективні агрегаційні запити. Освоїти методи фільтрації, групування, сортування та обробки масивів у MongoDB. Практично працювати з \$match, \$group, \$sort, \$unwind, \$lookup, \$project. Аналізувати продуктивність агрегацій та оптимізувати запити.

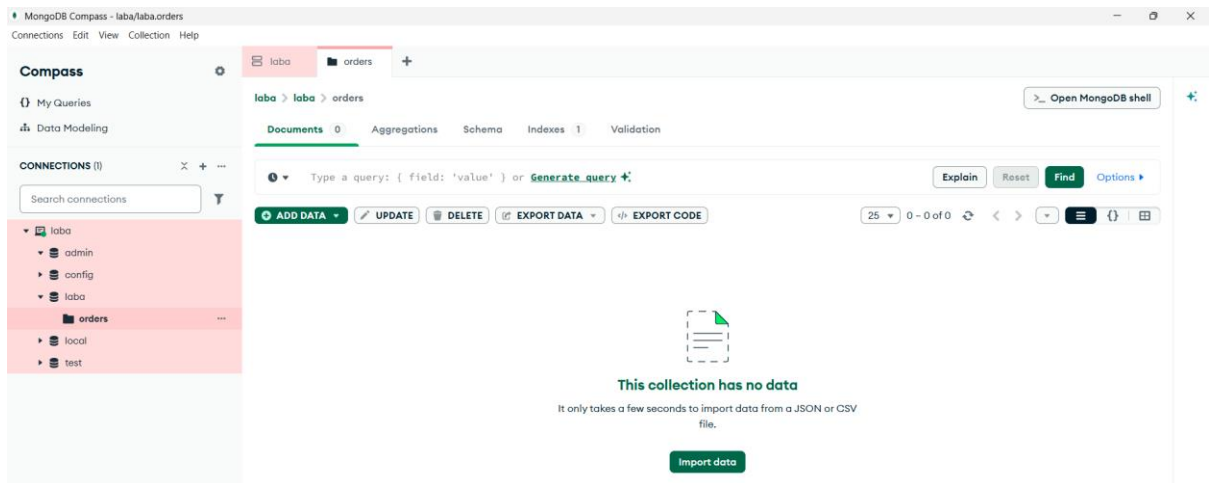
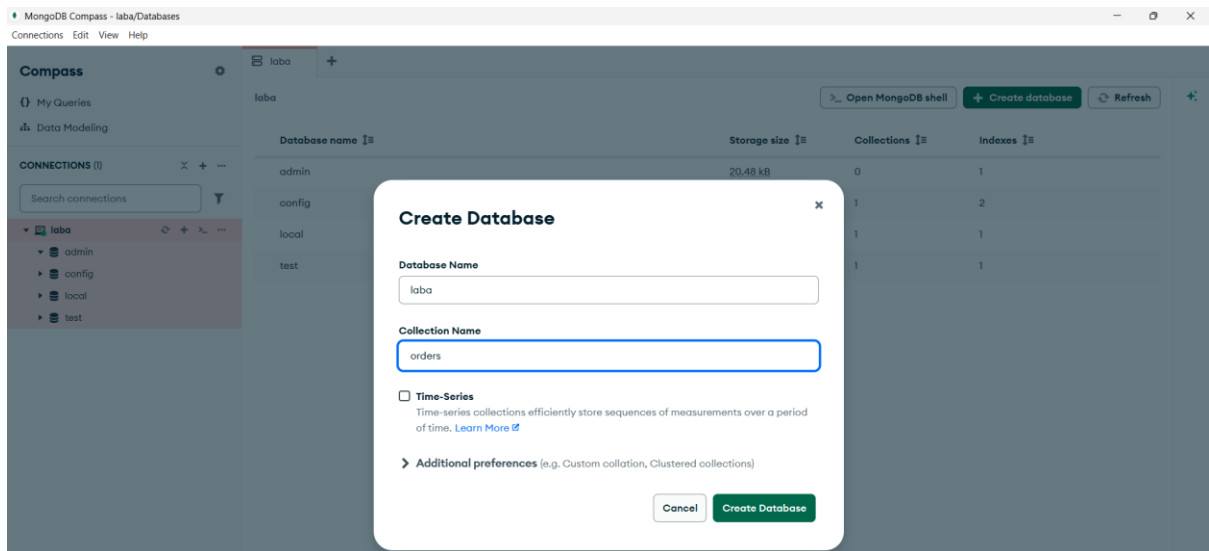
Хід роботи

Вихідні дані:

Для виконання лабораторної роботи потрібно створити наступні колекції в MongoDB:

1. Колекція orders

```
{
  "_id": ObjectId("..."),
  "orderId": "ORD001",
  "customerId": ObjectId("..."),
  "date": ISODate("2024-01-12"),
  "items": [
    { "product": "Laptop", "quantity": 1, "price": 1200 },
    { "product": "Mouse", "quantity": 2, "price": 50 }
  ],
  "status": "Completed"
}
```



mongosh: laba orders +

laba > laba > orders

Documents 1 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA UPDATE DELETE EXPORT DATA EXPORT CODE

```
{
  "_id": ObjectId('699b56ffe9da5b6941eb2fa1'),
  "orderId": "ORD001",
  "customerId": ObjectId('699b5779e9da5b6941eb2fa2'),
  "date": "2024-01-12T00:00:00.000+00:00",
  "items": Array (2)
    0: Object
      product: "Laptop"
      quantity: 1
      price: 1200
    1: Object
      product: "Mouse"
      quantity: 2
      price: 50
  "status": "Completed"
}
```

Успішно створена база даних **laba** та колекція **orders**.

2. Колекція customers

```
{
  "_id": ObjectId("..."),
  "name": "John Doe",
  "email": "john.doe@example.com",
  "city": "New York",
  "registeredAt": ISODate("2021-03-15")
}
```

```

> db.customers.insertOne({

  name: "John Doe",

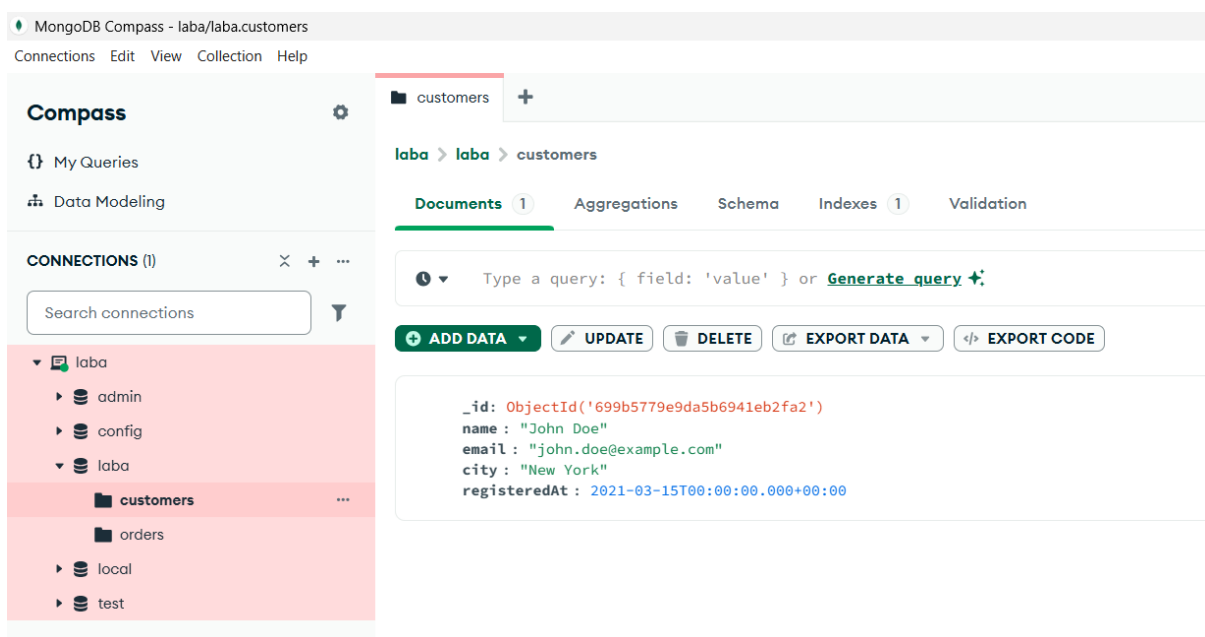
  email: "john.doe@example.com",

  city: "New York",

  registeredAt: new Date("2021-03-15")

})
< {
  acknowledged: true,
  insertedId: ObjectId('699b5779e9da5b6941eb2fa2')
}
laba>

```



Успішно створена колекція **customers**.

3. Колекція products

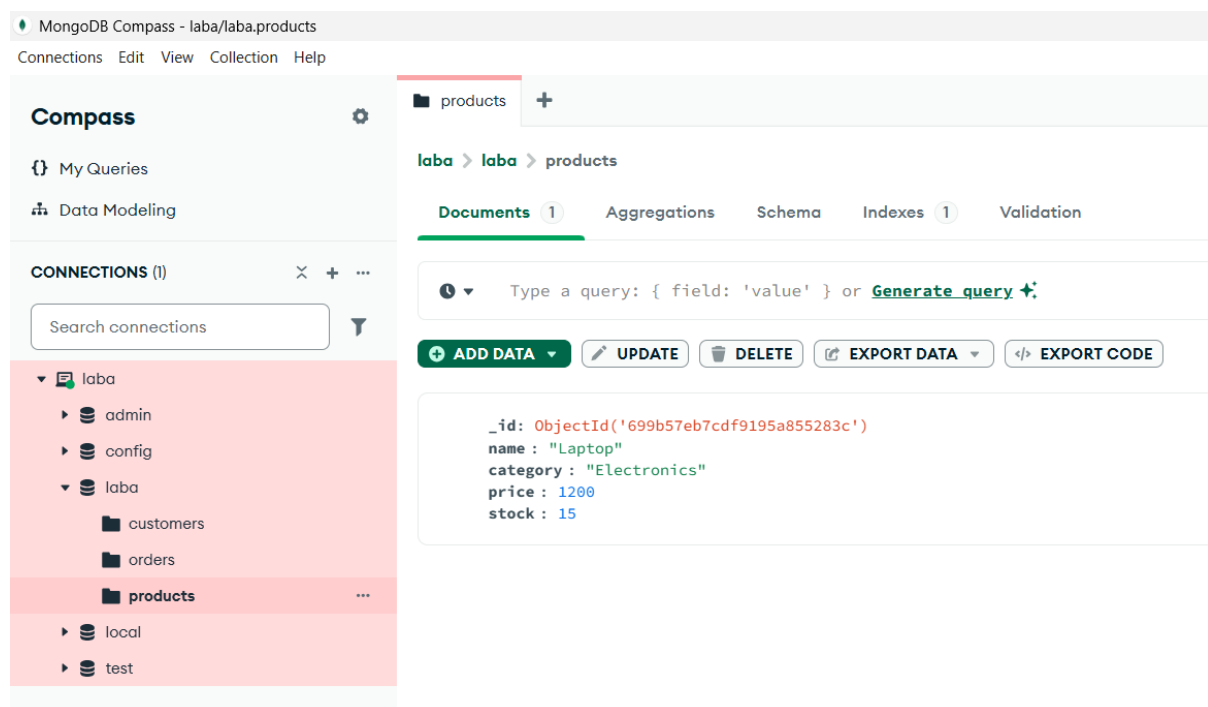
```

{
  "_id": ObjectId("..."),
  "name": "Laptop",

```

```
"category": "Electronics",  
"price": 1200,  
"stock": 15  
}
```

```
> use laba  
< switched to db laba  
> db.products.insertOne({  
  name: "Laptop",  
  category: "Electronics",  
  price: 1200,  
  stock: 15  
})  
< {  
  acknowledged: true,  
  insertedId: ObjectId('699b57eb7cdf9195a855283c')  
}  
laba >
```

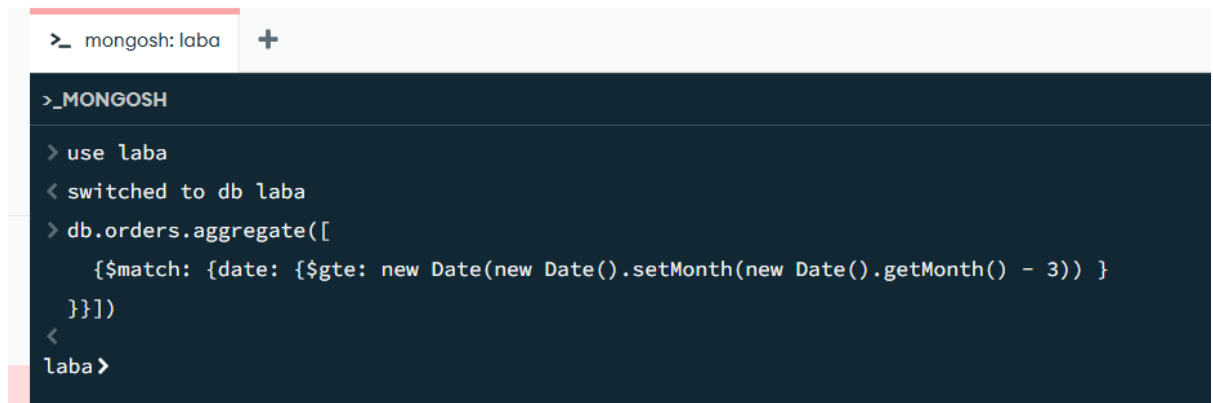


Успішно створена колекція **products**.

Завдання:

Частина 1: Базові агрегаційні операції

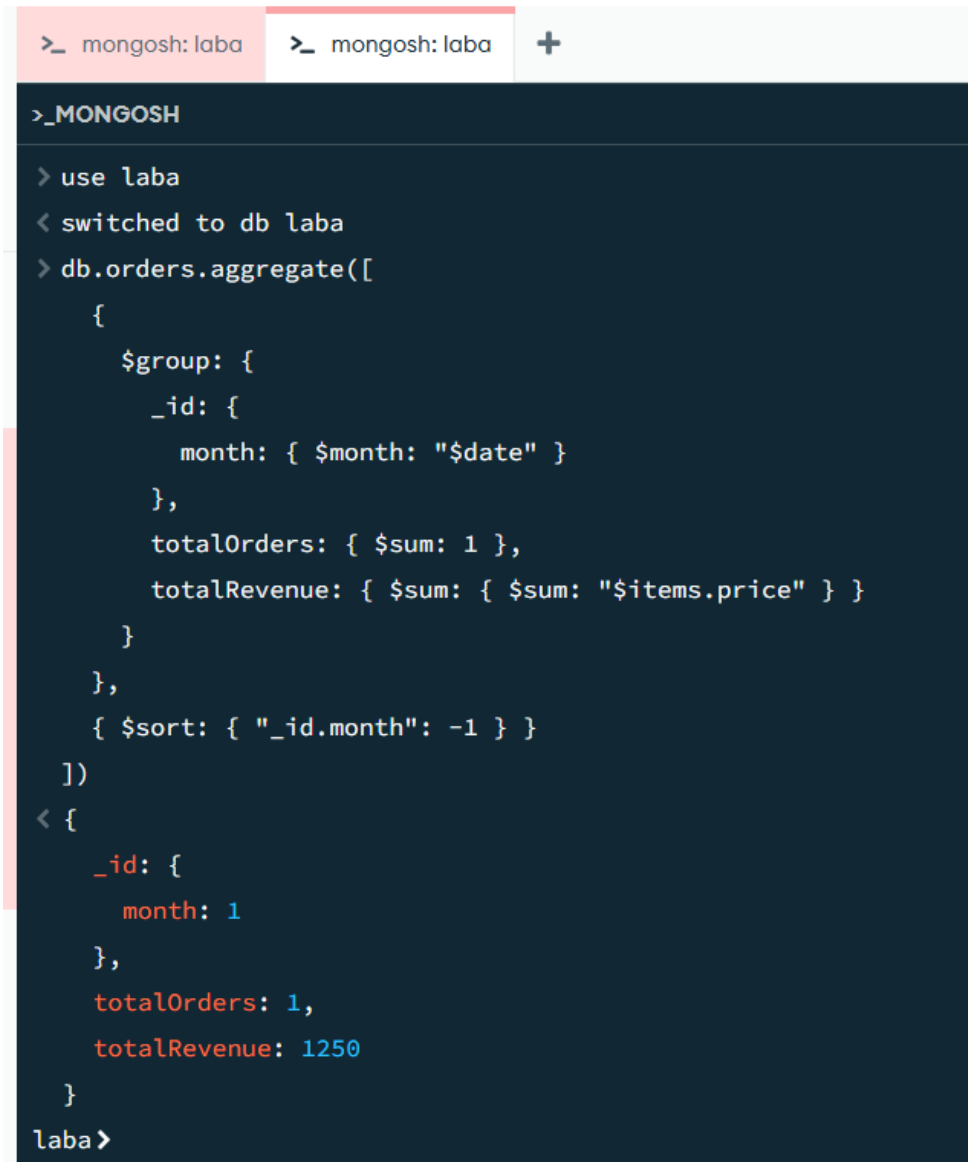
1. Відфільтруйте замовлення за останні 3 місяці



```
>_ mongosh: laba +
>_MONGOSH
> use laba
< switched to db laba
> db.orders.aggregate([
  {$match: {date: {$gte: new Date(new Date().setMonth(new Date().getMonth() - 3)) }
  }}])
<
laba>
```

Запит використовує оператор `$match` для фільтрації замовлень за останні три місяці від поточної дати. Оскільки дата наявного замовлення — 12.01.2024, а поточний рік — 2026, документ не входить до заданого часового діапазону. У результаті запит не повернув жодного запису, що свідчить про відсутність документів, які відповідають умові фільтрації.

2. Групування замовлень за місяцем

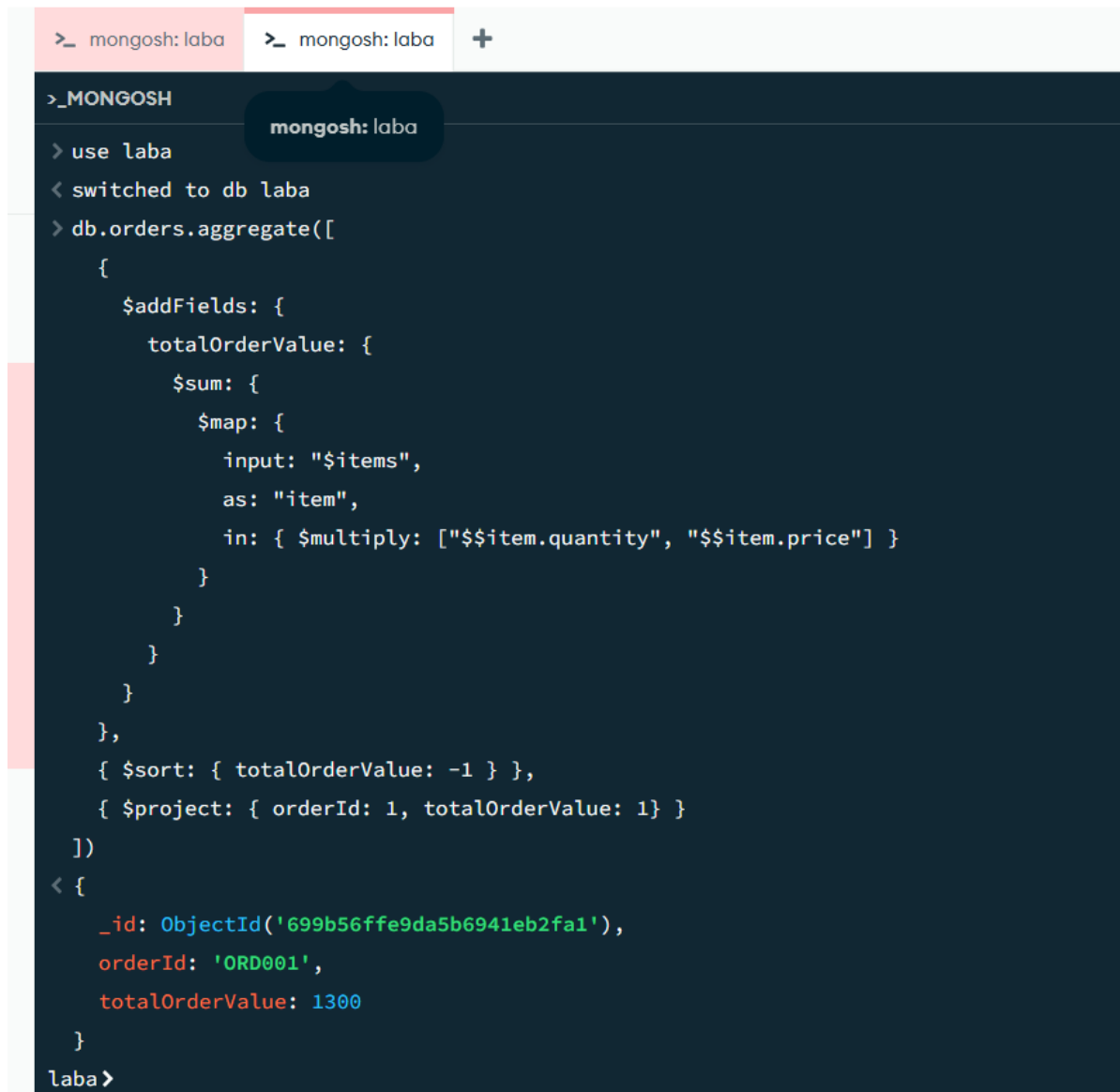


```
>_ mongosh: laba >_ mongosh: laba +
>_MONGOSH
> use laba
< switched to db laba
> db.orders.aggregate([
  {
    $group: {
      _id: {
        month: { $month: "$date" }
      },
      totalOrders: { $sum: 1 },
      totalRevenue: { $sum: { $sum: "$items.price" } }
    }
  },
  { $sort: { "_id.month": -1 } }
])
< {
  _id: {
    month: 1
  },
  totalOrders: 1,
  totalRevenue: 1250
}
laba>
```

Цей запит групує всі замовлення з колекції orders за місяцем, підраховуючи їх кількість (totalOrders: 1) та загальний дохід (totalRevenue: 1250).

Результат показує, що у січні 2024 року було виконано одне замовлення на суму 1250, після чого дані сортуються за спаданням місяця.

3. Сортування за сумою замовлення



```
>_ mongosh: laba >_ mongosh: laba +
>_MONGOSH
> use laba
< switched to db laba
> db.orders.aggregate([
  {
    $addFields: {
      totalOrderValue: {
        $sum: {
          $map: {
            input: "$items",
            as: "item",
            in: { $multiply: ["$$item.quantity", "$$item.price"] }
          }
        }
      }
    }
  },
  { $sort: { totalOrderValue: -1 } },
  { $project: { orderId: 1, totalOrderValue: 1 } }
])
< {
  _id: ObjectId('699b56ffe9da5b6941eb2fa1'),
  orderId: 'ORD001',
  totalOrderValue: 1300
}
laba>
```

Цей запит спочатку обчислює загальну вартість кожного замовлення шляхом перемноження ціни та кількості для кожного товару в масиві `items` за допомогою `$map`, а потім підсумовує ці значення через `$sum`. Далі запит сортує всі замовлення за спаданням отриманої суми (`totalOrderValue`) та виводить тільки поля `orderId` і `totalOrderValue`, де найбільше замовлення (`ORD001`) має вартість 1300.

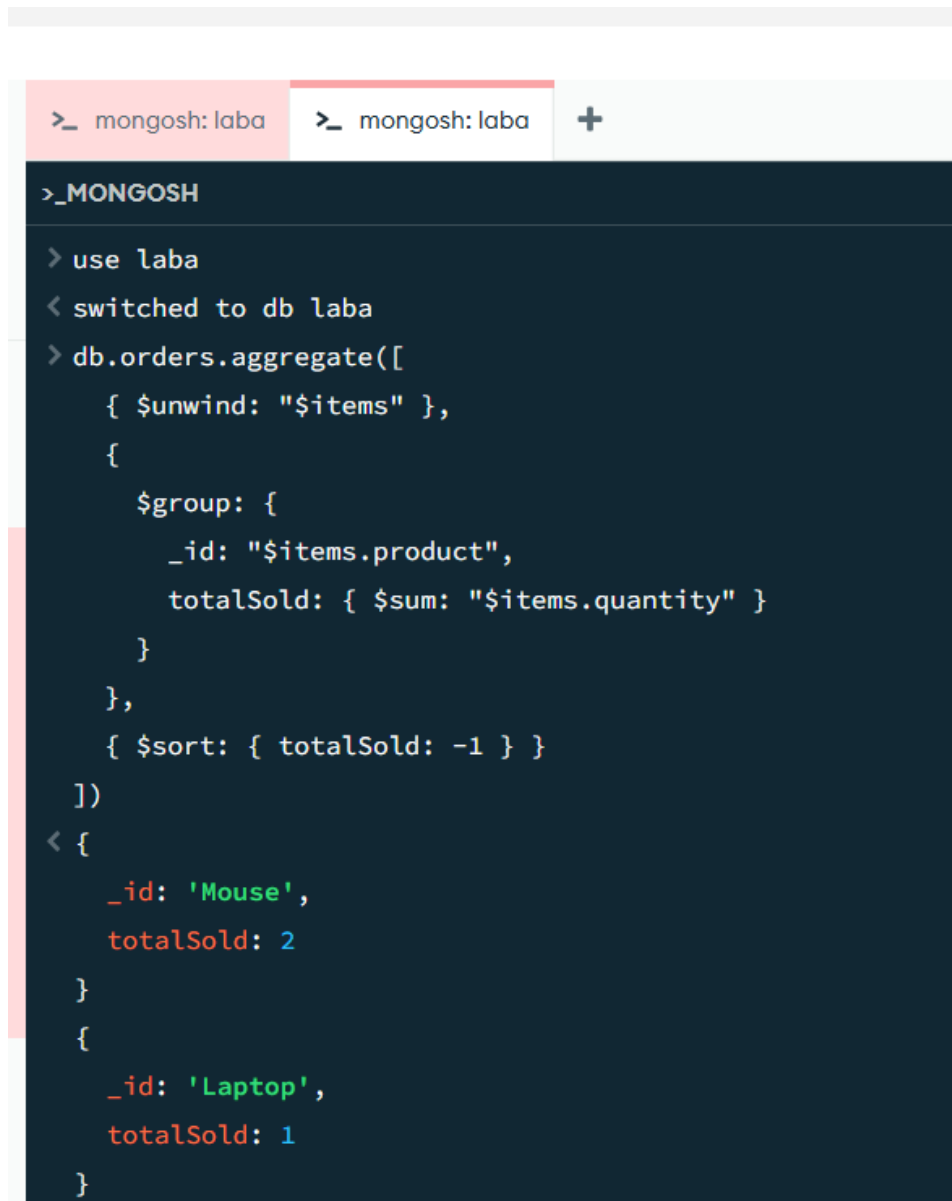
Частина 2: Робота з масивами

4. Розгорніть масив `items` у замовленнях

```
➤ mongosh: laba +
>_MONGOSH
> use laba
< switched to db laba
> db.orders.aggregate([
  { $unwind: "$items" }
])
< {
  _id: ObjectId('699b56ffe9da5b6941eb2fa1'),
  orderId: 'ORD001',
  customerId: ObjectId('699b5779e9da5b6941eb2fa2'),
  date: 2024-01-12T00:00:00.000Z,
  items: {
    product: 'Laptop',
    quantity: 1,
    price: 1200
  },
  status: 'Completed'
}
{
  _id: ObjectId('699b56ffe9da5b6941eb2fa1'),
  orderId: 'ORD001',
  customerId: ObjectId('699b5779e9da5b6941eb2fa2'),
  date: 2024-01-12T00:00:00.000Z,
  items: {
    product: 'Mouse',
    quantity: 2,
    price: 50
  },
  status: 'Completed'
}
laba> |
```

Цей запит використовує \$unwind для деструктуризації масиву items у замовленні ORD001, створюючи окремі документи для кожного елемента масиву. В результаті одне замовлення, яке містило два товари (Laptop та Mouse), було розділено на два самостійні документи з однаковою інформацією про замовлення, але різними об'єктами товарів у полі items.

5. Підрахуйте кількість проданих одиниць товарів

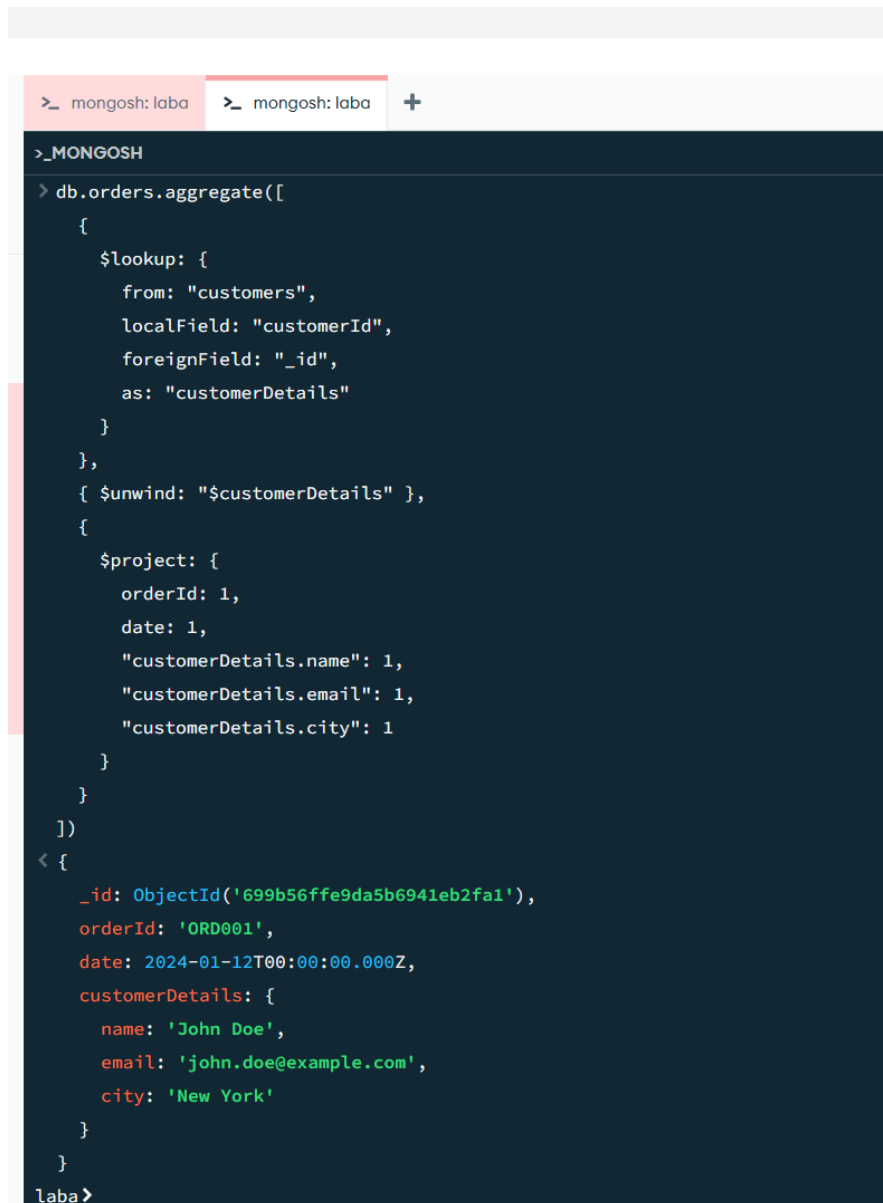


```
>_ mongosh: laba >_ mongosh: laba +
>_MONGOSH
> use laba
< switched to db laba
> db.orders.aggregate([
  { $unwind: "$items" },
  {
    $group: {
      _id: "$items.product",
      totalSold: { $sum: "$items.quantity" }
    }
  },
  { $sort: { totalSold: -1 } }
])
< {
  _id: 'Mouse',
  totalSold: 2
}
{
  _id: 'Laptop',
  totalSold: 1
}
```

Результат показує, що Mouse продано 2 штуки, а Laptop- 1.

Частина 3: З'єднання колекцій (\$lookup)

6. Отримання інформації про клієнтів у замовленнях



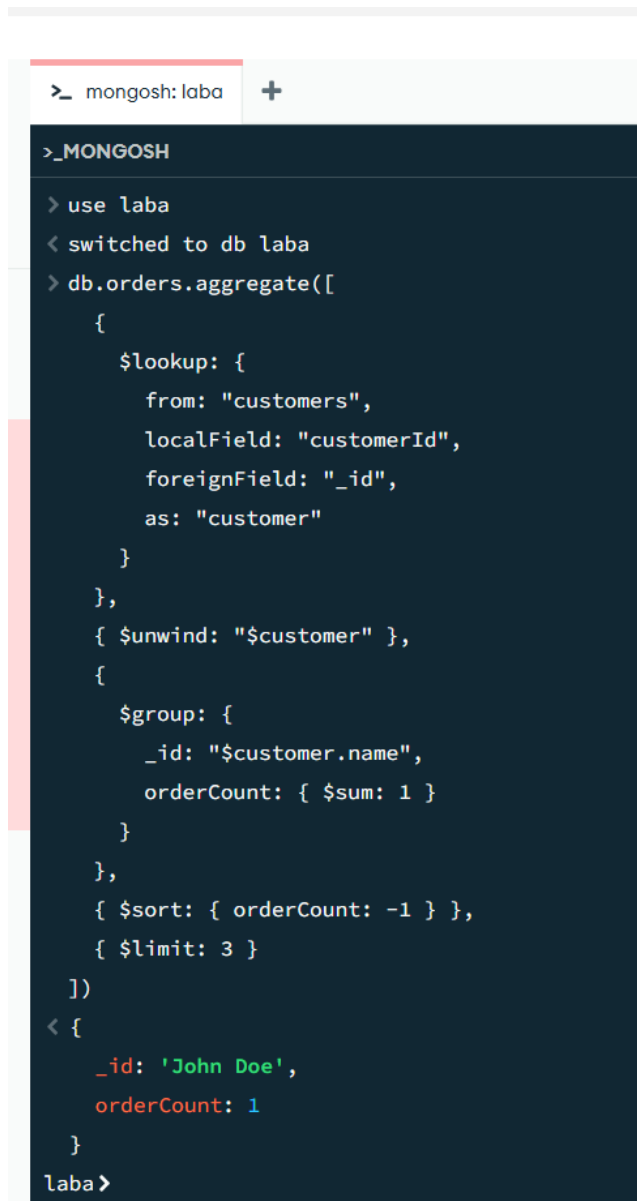
```
>_ mongosh: laba >_ mongosh: laba +
>_MONGOSH
> db.orders.aggregate([
  {
    $lookup: {
      from: "customers",
      localField: "customerId",
      foreignField: "_id",
      as: "customerDetails"
    }
  },
  { $unwind: "$customerDetails" },
  {
    $project: {
      orderId: 1,
      date: 1,
      "customerDetails.name": 1,
      "customerDetails.email": 1,
      "customerDetails.city": 1
    }
  }
])
< {
  _id: ObjectId('699b56ffe9da5b6941eb2fa1'),
  orderId: 'ORD001',
  date: 2024-01-12T00:00:00.000Z,
  customerDetails: {
    name: 'John Doe',
    email: 'john.doe@example.com',
    city: 'New York'
  }
}
```

Цей запит об'єднує дані з двох колекцій - orders та customers, за допомогою \$lookup, яка знаходить відповідного клієнта для кожного замовлення на основі збігу customerId та _id.

Після цього \$unwind перетворює отриманий масив з даними клієнта на звичайний об'єкт, а \$project залишає у фінальному результаті тільки номер замовлення, дату та ім'я, email і місто проживання клієнта.

У результаті кожне замовлення відображається разом з детальною інформацією про клієнта.

7. Визначте найбільш активних клієнтів



```
> mongosh: laba +
> _MONGOSH
> use laba
< switched to db laba
> db.orders.aggregate([
  {
    $lookup: {
      from: "customers",
      localField: "customerId",
      foreignField: "_id",
      as: "customer"
    }
  },
  { $unwind: "$customer" },
  {
    $group: {
      _id: "$customer.name",
      orderCount: { $sum: 1 }
    }
  },
  { $sort: { orderCount: -1 } },
  { $limit: 3 }
])
< {
  _id: 'John Doe',
  orderCount: 1
}
laba>
```

Цей запит визначає найбільш активних клієнтів на основі кількості зроблених ними замовлень.

У результаті John Дое зробив одне замовлення і поки що є єдиним активним клієнтом у цій базі даних.

Частина 4: Оптимізація запитів

8. Перевірте продуктивність запиту

> mongosh: laba



> _MONGOSH

```
> use laba
< switched to db laba
> db.orders.aggregate([
  {
    $match: {
      date: { $gte: new Date("2024-01-01") }
    }
  },
  {
    $group: {
      _id: { month: { $month: "$date" }, year: { $year: "$date" } },
      totalAmount: { $sum: { $sum: "$items.price" } }
    }
  }
]).explain("executionStats")
```

1. **\$match** фільтрує замовлення за датою.
2. **\$group** агрегує дані за місяцем і сумує ціну всіх товарів.
3. **.explain("executionStats")** показує статистику виконання:
 - a. **nReturned** — кількість документів у результаті.
 - b. **totalDocsExamined** — кількість документів, які MongoDB переглянув.
 - c. **executionTimeMillis** — час виконання запиту.

```
< {
  explainVersion: '1',
  stages: [
    {
      '$cursor': {
        queryPlanner: {
          namespace: 'laba.orders',
          parsedQuery: {
            date: {
              '$gte': 2024-01-01T00:00:00.000Z
            }
          },
        },
        indexFilterSet: false,
        queryHash: '35D42CE6',
        planCacheShapeHash: '35D42CE6',
        planCacheKey: '65C9966B',
        optimizationTimeMillis: 2,
        maxIndexedOrSolutionsReached: false,
        maxIndexedAndSolutionsReached: false,
        maxScansToExplodeReached: false,
        prunedSimilarIndexes: false,
        winningPlan: {
          isCached: false,
          stage: 'PROJECTION_DEFAULT',
          transformBy: {
            date: 1,
            'items.price': 1,
            _id: 0
          }
        }
      }
    }
  ]
}
```

```
    _id: 0
  },
  inputStage: {
    stage: 'COLLSCAN',
    filter: {
      date: {
        '$gte': 2024-01-01T00:00:00.000Z
      }
    },
    direction: 'forward'
  }
},
rejectedPlans: []
},
executionStats: {
  executionSuccess: true,
  nReturned: 1,
  executionTimeMillis: 10,
  totalKeysExamined: 0,
  totalDocsExamined: 1,
  executionStages: {
    isCached: false,
    stage: 'PROJECTION_DEFAULT',
    nReturned: 1,
    executionTimeMillisEstimate: 0,
    works: 2,
    advanced: 1,
    needTime: 0,
    needYield: 0,
    saveState: 2,
```

```

    saveState: 2,
    restoreState: 1,
    isEOF: 1,
    transformBy: {
      date: 1,
      'items.price': 1,
      _id: 0
    },
    inputStage: {
      stage: 'COLLSCAN',
      filter: {
        date: {
          '$gte': 2024-01-01T00:00:00.000Z
        }
      }
    },
    nReturned: 1,
    executionTimeMillisEstimate: 0,
    works: 2,
    advanced: 1,
    needTime: 0,
    needYield: 0,
    saveState: 2,
    restoreState: 1,
    isEOF: 1,
    direction: 'forward',
    docsExamined: 1
  }
}
}
},

```

1. **nReturned: 1** — запит повернув 1 документ.
2. **executionTimeMillis: 0** — дуже швидко виконався.
3. **totalDocsExamined: 1** — перевірено лише 1 документ, що ефективно.

```
    },
    nReturned: 1,
    executionTimeMillisEstimate: 6
  },
  {
    '$group': {
      _id: {
        month: {
          '$month': {
            date: '$date'
          }
        },
        year: {
          '$year': {
            date: '$date'
          }
        }
      },
      totalAmount: {
        '$sum': {
          '$sum': [
            '$items.price'
          ]
        }
      },
      '$willBeMerged': false
    },
    maxAccumulatorMemoryUsageBytes: {
      totalAmount: 208
    },
  },
```

```
    },
    totalOutputDataSizeBytes: 474,
    usedDisk: false,
    spills: 0,
    spilledDataStorageSize: 0,
    spilledBytes: 0,
    spilledRecords: 0,
    nReturned: 1,
    executionTimeMillisEstimate: 6
  }
],
queryShapeHash: '5ED5D5A65FD2E30A7028003FA1723FAAD1AA3DEC3BC1BC53ABF3F3A491607F9D',
serverInfo: {
  host: 'DESKTOP-2RG0T4B',
  port: 27017,
  version: '8.2.4',
  gitVersion: '60692d74054a0bc73f181179e747815825de38ae'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
```

```

    },
    command: {
      aggregate: 'orders',
      pipeline: [
        {
          '$match': {
            date: {
              '$gte': 2024-01-01T00:00:00.000Z
            }
          }
        }
      ],
    },
    {
      '$group': {
        _id: {
          month: {
            '$month': '$date'
          },
          year: {
            '$year': '$date'
          }
        },
        totalAmount: {
          '$sum': {
            '$sum': '$items.price'
          }
        }
      }
    }
  ],
  cursor: {},
  '$db': 'laba'

```

```

    },
    ok: 1
  }
laba>

```

9. Оптимізуйте агрегаційний запит

```
>_ mongosh: laba +
>_MONGOSH
> use laba
< switched to db laba
> db.orders.aggregate([
  {
    $match: {
      date: { $gte: new Date("2024-01-01") }
    }
  },
  {
    $unwind: "$items"
  },
  {
    $group: {
      _id: { month: { $month: "$date" }, year: { $year: "$date" } },
      totalAmount: { $sum: { $multiply: ["$items.quantity", "$items.price"] } },
      totalItems: { $sum: "$items.quantity" }
    }
  },
  {
    $sort: { totalAmount: -1 }
  }
]).explain("executionStats")
```

1. **\$match** на початку зменшує обсяг даних.
2. **\$unwind** дозволяє коректно рахувати кількість одиниць товару.
3. **\$group** підраховує і суму, і кількість.
4. **\$sort** після групування, щоб не сортувати зайві документи.
5. Використання індексу на date сильно прискорює \$match.

```
{
  explainVersion: '1',
  stages: [
    {
      '$cursor': {
        queryPlanner: {
          namespace: 'laba.orders',
          parsedQuery: {
            date: {
              '$gte': 2024-01-01T00:00:00.000Z
            }
          },
        },
        indexFilterSet: false,
        queryHash: '1F494B1D',
        planCacheShapeHash: '1F494B1D',
        planCacheKey: 'D0FCC07C',
        optimizationTimeMillis: 1,
        maxIndexedOrSolutionsReached: false,
        maxIndexedAndSolutionsReached: false,
        maxScansToExplodeReached: false,
        prunedSimilarIndexes: false,
        winningPlan: {
          isCached: false,
          stage: 'PROJECTION_SIMPLE',
          transformBy: {
            date: 1,
            items: 1,
            _id: 0
          },
        },
        inputStage: {
```

```
    },
    inputStage: {
      stage: 'COLLSCAN',
      filter: {
        date: {
          '$gte': '2024-01-01T00:00:00.000Z'
        }
      },
      direction: 'forward'
    }
  },
  rejectedPlans: []
},
executionStats: {
  executionSuccess: true,
  nReturned: 1,
  executionTimeMillis: 16,
  totalKeysExamined: 0,
  totalDocsExamined: 1,
  executionStages: {
    isCached: false,
    stage: 'PROJECTION_SIMPLE',
    nReturned: 1,
    executionTimeMillisEstimate: 0,
    works: 2,
    advanced: 1,
    needTime: 0,
    needYield: 0,
    saveState: 2,
    restoreState: 1,
    isEOF: 1,
  }
}
```

_MONGOSH

```
    restoreState: 1,
    isEOF: 1,
    transformBy: {
      date: 1,
      items: 1,
      _id: 0
    },
    inputStage: {
      stage: 'COLLSCAN',
      filter: {
        date: {
          '$gte': 2024-01-01T00:00:00.000Z
        }
      },
      nReturned: 1,
      executionTimeMillisEstimate: 0,
      works: 2,
      advanced: 1,
      needTime: 0,
      needYield: 0,
      saveState: 2,
      restoreState: 1,
      isEOF: 1,
      direction: 'forward',
      docsExamined: 1
    }
  }
}
},
nReturned: 1,
executionTimeMillisEstimate: 6
```

>_MONGOSH

```
    }
  }
}
},
nReturned: 1,
executionTimeMillisEstimate: 6
},
{
  '$unwind': {
    path: '$items'
  },
  nReturned: 2,
  executionTimeMillisEstimate: 6
},
{
  '$group': {
    _id: {
      month: {
        '$month': {
          date: '$date'
        }
      },
      year: {
        '$year': {
          date: '$date'
        }
      }
    },
    totalAmount: {
      '$sum': {
        '$multiply': [
```

>_MONGOSH

```
    ]
  }
},
totalItems: {
  '$sum': '$items.quantity'
},
'$willBeMerged': false
},
maxAccumulatorMemoryUsageBytes: {
  totalAmount: 208,
  totalItems: 208
},
totalOutputDataSizeBytes: 506,
usedDisk: false,
spills: 0,
spilledDataStorageSize: 0,
spilledBytes: 0,
spilledRecords: 0,
nReturned: 1,
executionTimeMillisEstimate: 6
},
{
  '$sort': {
    sortKey: {
      totalAmount: -1
    }
  },
  totalDataSizeSortedBytesEstimate: 514,
  usedDisk: false,
  spills: 0,
  spilledDataStorageSize: 0,
```

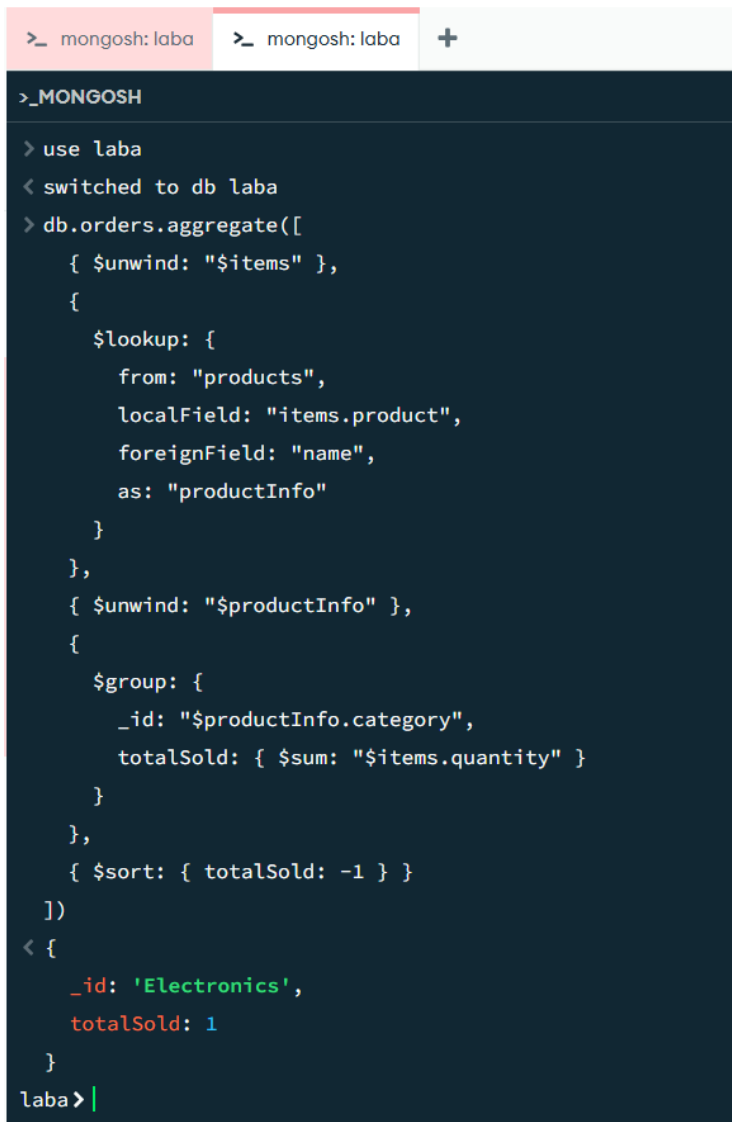
```
],
queryShapeHash: 'DE860F2B9681684A62A4893A8136257C9486DEE00580548F842F148D6AD20711',
serverInfo: {
  host: 'DESKTOP-2RGOT4B',
  port: 27017,
  version: '8.2.4',
  gitVersion: '60692d74054a0bc73f181179e747815825de38ae'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
command: {
  aggregate: 'orders',
  pipeline: [
    {
      '$match': {
        date: {
          '$gte': 2024-01-01T00:00:00.000Z
        }
      }
    }
  ]
},
{
```

```
{
  '$group': {
    _id: {
      month: {
        '$month': '$date'
      },
      year: {
        '$year': '$date'
      }
    },
    totalAmount: {
      '$sum': {
        '$multiply': [
          '$items.quantity',
          '$items.price'
        ]
      }
    },
    totalItems: {
      '$sum': '$items.quantity'
    }
  },
  {
    '$sort': {
      totalAmount: -1
    }
  }
],
cursor: {},
'$db': 'laba'
```

```
    },
    totalItems: {
      '$sum': '$items.quantity'
    }
  },
  {
    '$sort': {
      totalAmount: -1
    }
  }
],
cursor: {},
'$db': 'laba'
},
ok: 1
}
laba> |
```

Додаткові завдання:

- 10. Визначте категорії товарів із найбільшою кількістю продажів. Використайте \$group для підрахунку загальної кількості проданих товарів за категоріями. Відсортуйте результат за спаданням.**



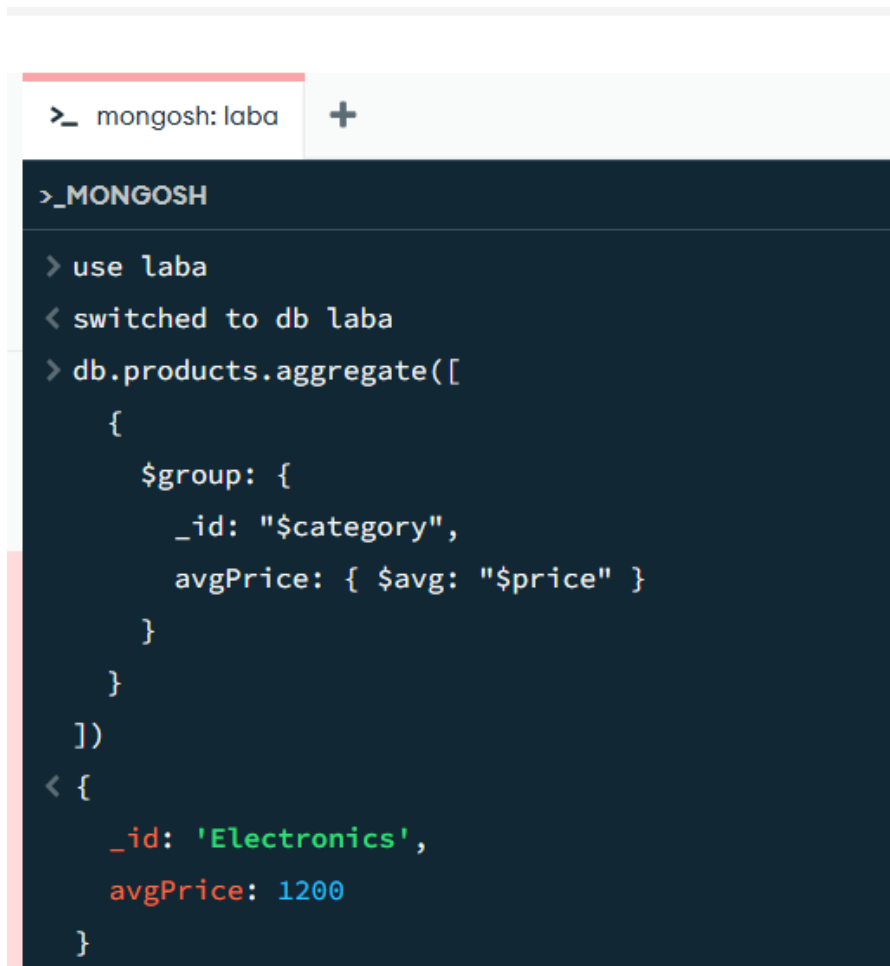
```
>_MONGOSH
> use laba
< switched to db laba
> db.orders.aggregate([
  { $unwind: "$items" },
  {
    $lookup: {
      from: "products",
      localField: "items.product",
      foreignField: "name",
      as: "productInfo"
    }
  },
  { $unwind: "$productInfo" },
  {
    $group: {
      _id: "$productInfo.category",
      totalSold: { $sum: "$items.quantity" }
    }
  },
  { $sort: { totalSold: -1 } }
])
< {
  _id: 'Electronics',
  totalSold: 1
}
laba>|
```

Цей запит визначає, яка категорія товарів має найбільшу кількість продажів. За допомогою `$unwind` розгортається масив `items` у замовленнях, щоб кожен товар став окремим документом. Через `$lookup` до кожного товару приєднується інформація з колекції `products` за назвою продукту, а `$unwind` розгортає отримані дані. Далі `$group` групує документи за категорією товару і підсумовує кількість проданих одиниць (`totalSold`) за допомогою `$sum`. `$sort` впорядковує результати за спаданням кількості продажів.

У результаті категорія `Electronics` має одну продану одиницю - це і є найбільша кількість продажів.

11.Розрахуйте середню ціну товарів у кожній категорії.

Використайте \$group для підрахунку середньої ціни товарів у кожній категорії.



```
> mongosh: laba +
> _MONGOSH
> use laba
< switched to db laba
> db.products.aggregate([
  {
    $group: {
      _id: "$category",
      avgPrice: { $avg: "$price" }
    }
  }
])
< {
  _id: 'Electronics',
  avgPrice: 1200
}
```

Цей запит групує товари з колекції products за категоріями та обчислює середню ціну товарів у кожній категорії.

Результат показує, що в категорії Electronics середня ціна товарів становить 1200.

12.Знайдіть користувачів, які зробили більше одного замовлення.

Використайте \$group і \$match, щоб знайти клієнтів, які мали більше одного замовлення.

```
>_ mongosh: laba +
>_MONGOSH
> use laba
< switched to db laba
> db.orders.aggregate([
  { $unwind: "$items" },
  {
    $lookup: {
      from: "products",
      localField: "items.product",
      foreignField: "name",
      as: "productInfo"
    }
  },
  { $unwind: "$productInfo" },
  {
    $group: {
      _id: "$productInfo.category",
      totalSold: { $sum: "$items.quantity" },
      totalRevenue: { $sum: { $multiply: ["$items.quantity", "$items.price"] } }
    }
  },
  { $sort: { totalSold: -1 } }
])
< {
  _id: 'Electronics',
  totalSold: 1,
  totalRevenue: 1200
}
laba>
```

Спочатку розгортається масив items у документах замовлень, потім через \$lookup приєднуються дані з колекції products за назвою товару, далі групуються результати за категорією товару, при цьому обчислюючи загальну кількість проданих одиниць totalSold та загальний дохід totalRevenue як суму добутків quantity на price. Далі отримані групи сортуються за спаданням totalSold.

У результаті отримується наразі лише одна категорія Electronics з одним проданим товаром та доходом 1200.

Висновок

У ході виконання лабораторної роботи реалізувала фільтрацію даних за датою, групування замовлень за місяцем, підрахунок загального доходу та кількості проданих товарів, роботу з масивами через \$unwind, а також з'єднання колекцій за допомогою \$lookup.

Також приділила увагу аналізу продуктивності запитів та їх оптимізації. У процесі дослідження було з'ясовано, що розміщення \$match на початку пайплайну, використання індексів та правильний порядок стадій значно впливають на швидкість виконання запиту та кількість оброблених документів.

Під час роботи складність полягала у правильному обчисленні загальної вартості замовлень та коректній роботі з масивами вкладених документів.

Завдяки використанню \$unwind, \$multiply та \$sum ці завдання були успішно реалізовані.